

Resultado de aprendizaje evaluado: el estudiante *diseña e implementa un servicio web REST del lado del servidor con Spring Boot 3 que expone recursos paginados bajo un contrato JSON uniforme, valida la entrada en el servidor, aplica borrado logico, integra el patron cache-aside con Redis y protege sus operaciones con autentificacion y autorizacion basadas en JSON Web Token* [1, 2, 3, 4].

Aporte al perfil de egreso. El examen integra los contenidos de las Unidades III (persistencia, patrones de arquitectura y cache) y IV (servicios REST y seguridad de aplicaciones) de la asignatura. Su dominio es prerequisite directo del Proyecto Fin de Curso (PFC), pues consolida la construccion de una API verificable, segura y reproducible por terceros [5].

Modalidad y escala. Individual. La calificacion se expresa sobre 100 y su equivalente sobre 10 se obtiene como $\text{Nota}_{10} = \text{Nota}_{100}/10$. Se aprueba con nota igual o superior a 7.0/10, conforme al reglamento academico UTEQ.

1. Criterios de piso (excluyentes)

Los criterios de piso son **condiciones de admision**: si cualquiera de ellos no se cumple, la calificacion del examen es **CERO** (0/10) y no se evalua ningun criterio de la rubrica. Esta separacion entre condiciones de admision y criterios de valor es practica estandar en la evaluacion de artefactos reproducibles [6].

1. **P1 — Caratula PDF en el SGA con la URL del repositorio (obligatorio).** El estudiante sube al LMS (SGA) un documento PDF con caratula de identificacion (apellidos y nombres, cedula, paralelo, asignatura, fecha del examen) que muestre claramente, **en una sola linea**, la URL del repositorio donde se encuentra el trabajo desarrollado. Si la URL esta rota, el repositorio no es accesible desde una sesion anonima, o no se cumple esta directriz, la calificacion es **CERO**.
2. **P2 — Reproducibilidad del PDF del informe desde el repositorio (obligatorio).** El repositorio debe incluir un informe tecnico en LaTeX (docs/informe/informe.tex con su .bib). Si el PDF no se genera correctamente a partir del LaTeX mediante clonacion del repositorio y compilacion del archivo .tex, la calificacion es **CERO**. Las instrucciones de compilacion (compilador, orden de comandos, archivo principal, dependencias) deben estar documentadas en el archivo README.md del repositorio.
3. **P3 — Aplicacion ejecutable en una clonacion limpia.** El repositorio debe permitir *clonar, construir y ejecutar* la API completa desde una maquina limpia siguiendo estrictamente el README.md, preferentemente con un solo comando (`docker compose up -d -build`) que levante la aplicacion, PostgreSQL y Redis. Si la aplicacion no arranca siguiendo el README, la parte de ejecucion no es verificable y aplica la clausula de piso.
4. **P4 — Autoria trazable.** El historial git log debe evidenciar commits del propio estudiante con su identidad. La suplantacion de autoria (git config manipulado) o la ausencia total de historial propio activa la clausula de piso.

2. Planteamiento del problema

2.1 Contexto

El Municipio de Quevedo necesita digitalizar el control de productos del mercado central. El equipo de TI le asigna a usted desarrollar el **modulo de gestion de inventario** como un servicio web REST construido con **Spring Boot 3** (Java 21 LTS). El servicio expone recursos de **Producto**, protege sus operaciones con **JSON Web Token** [3] y acelera las lecturas con **cache-aside sobre Redis** [4]. El trabajo se entrega en un repositorio GitHub publico, autocontenido y ejecutable desde una clonacion limpia.

2.2 Entidad de dominio: Producto

Atributo	Tipo (PostgreSQL)	Restricciones
id	BIGSERIAL	clave primaria autonumerica.
nombre	VARCHAR(100)	obligatorio, no vacio.
categoria	VARCHAR(50)	obligatorio.
stock	INTEGER	obligatorio, valor minimo 0.
precio	DECIMAL(10,2)	obligatorio, valor minimo mayor o igual a 0.01.
activo	BOOLEAN DEFAULT TRUE	la eliminacion es <i>logica</i> (soft delete).
creadoEn	TIMESTAMPTZ	fecha-hora de creacion en zona horaria.

2.3 Requisitos funcionales exigidos

El servicio debe implementar los cinco requisitos siguientes; cada uno se corresponde con un criterio de la rubrica de la seccion 4.

1. **Listado paginado.** Endpoint GET /api/v1/productos que devuelve la lista de productos paginada mediante los parametros `page`, `size` y `sort`. La respuesta usa la envoltura JSON estandar `{success, data, message, meta}`, donde `meta` incluye la informacion de paginacion (pagina actual, tamano, total de elementos y total de paginas).
2. **Creacion validada.** Endpoint POST /api/v1/productos que crea un producto nuevo. El DTO de entrada se valida con `@Valid` y anotaciones `@NotBlank`, `@Min` y `@DecimalMin`. Si la validacion falla, la API devuelve HTTP 400 con la lista de errores por campo. Se recomienda estructurar la respuesta de error segun el contrato Problem Details [7].
3. **Eliminacion logica.** Endpoint DELETE /api/v1/productos/{id} que realiza *soft delete* (cambia `activo = false`; nunca borrado fisico). Si el `id` no existe, la API devuelve HTTP 404 con un mensaje descriptivo.
4. **Cache-aside con Redis.** Aplicar el patron cache-aside con Redis mediante `@Cacheable` en el listado de productos; al crear o eliminar, invalidar la entrada de cache con `@CacheEvict` [4].
5. **Seguridad con JWT.** Proteger los tres endpoints con JSON Web Token [3]: el GET requiere rol `ROLE_USER`; el POST y el DELETE requieren `ROLE_ADMIN`. Un intento sin token debe devolver HTTP 401; un intento con token valido pero rol insuficiente debe devolver HTTP 403 [8].

2.4 Contrato de respuesta y ejemplos

La envoltura de respuesta es uniforme para exito y error. El campo `meta` es `null` en operaciones que no paginan.

```
GET /api/v1/productos?page=0&size=10&sort=nombre,asc -> 200 OK
{
  "success": true,
  "data": [
    { "id": 1, "nombre": "Arroz flor", "categoria": "Granos",
      "stock": 120, "precio": 0.95, "activo": true,
      "creadoEn": "2026-07-24T09:15:00-05:00" }
  ],
  "message": "Listado obtenido correctamente",
  "meta": { "page": 0, "size": 10, "totalElements": 57, "totalPages": 6 }
}

POST /api/v1/productos (body invalido) -> 400 Bad Request
{
  "success": false,
  "data": null,
  "message": "Error de validacion",
  "errors": [
    { "field": "nombre", "message": "no debe estar vacio" },
    { "field": "precio", "message": "debe ser mayor o igual a 0.01" }
  ]
}

DELETE /api/v1/productos/9999 (inexistente) -> 404 Not Found
GET sin token -> 401 Unauthorized
POST con token de ROLE_USER -> 403 Forbidden
```

3. Guia de desarrollo — pasos hacia el resultado de aprendizaje

Los pasos siguientes conectan cada requisito con los contenidos de las Unidades III y IV y con una arquitectura por capas limpia (controlador delgado, servicio con reglas de negocio, repositorio como unico lugar de acceso a datos) [9, 10].

Paso 1. Iniciar el proyecto Spring Boot 3

Generar el proyecto en `start.spring.io` con Java 21 LTS y las dependencias *Spring Web*, *Spring Data JPA*, *Validation*, *Spring Security*, *Spring Data Redis* y el driver *PostgreSQL* [2]. Definir el esquema y el *seed* para que arranquen desde una base vacia.

```
CREATE TABLE productos (
  id          BIGSERIAL      PRIMARY KEY,
  nombre      VARCHAR(100)   NOT NULL,
  categoria   VARCHAR(50)    NOT NULL,
  stock       INTEGER        NOT NULL CHECK (stock >= 0),
  precio      DECIMAL(10,2)  NOT NULL CHECK (precio >= 0.01),
  activo      BOOLEAN        NOT NULL DEFAULT TRUE,
  creado_en   TIMESTAMPTZ    NOT NULL DEFAULT now()
);
```

Paso 2. Modelar entidad, DTO y contrato de respuesta

Mapear `Producto` con JPA. Separar el DTO de entrada (`ProductoRequest`) del DTO de salida (`ProductoResponse`). Definir una envoltura generica `ApiResponse<T>` con los campos `success`, `data`, `message` y `meta`, de modo que todas las respuestas del servicio compartan la misma forma.

Paso 3. Endpoint GET paginado

Recibir `Pageable` con `page`, `size` y `sort`; consultar solo los productos con `activo = true`; poblar `meta` con `totalElements` y `totalPages` a partir del objeto `Page` devuelto por Spring Data JPA. El controlador delega en el servicio y este en el repositorio; nunca hay logica de acceso a datos en el controlador [10].

Paso 4. Endpoint POST con validacion en servidor

Anotar el `ProductoRequest` con `@NotBlank` (`nombre`, `categoria`), `@Min(0)` (`stock`) y `@DecimalMin(value="0.01")` (`precio`). En el controlador, anteponer `@Valid` al parametro del cuerpo. Capturar `MethodArgumentNotValidException` con un `@RestControllerAdvice` que construya la lista de errores por campo y devuelva HTTP 400, preferentemente bajo el contrato `Problem Details` [7]. La validacion de cliente es ayuda al usuario; el servidor es la fuente de verdad.

Paso 5. Endpoint DELETE con soft delete y 404

Buscar el producto por `id`; si no existe, lanzar una excepcion de recurso no encontrado que el `@RestControllerAdvice` traduzca a HTTP 404 con mensaje descriptivo. Si existe, fijar `activo = false` y persistir; jamas ejecutar DELETE fisico sobre la fila.

Paso 6. Cache-aside con Redis

Anotar el metodo de listado con `@Cacheable(value="productos", key="...")` para servir desde Redis cuando la clave exista (*cache hit*) y consultar la base solo en el *cache miss*. Anotar creacion y eliminacion con `@CacheEvict(value="productos", allEntries=true)` para invalidar el listado cacheado y evitar servir datos obsoletos [4]. Configurar un TTL prudente para el espacio de nombres de cache.

Paso 7. Seguridad con JWT (autenticacion y autorizacion)

Configurar *Spring Security* con un filtro que valide el token *Bearer* en cada peticion [3]. Exigir `ROLE_USER` para `GET /api/v1/productos` y `ROLE_ADMIN` para `POST` y `DELETE`. Configurar el `AuthenticationEntryPoint` para responder 401 ante peticiones sin token o con token invalido, y el `AccessDeniedHandler` para responder 403 ante token valido con rol insuficiente [8]. Nunca exponer el secreto de firma en el repositorio: usar `.env.example` con valores ficticios.

Paso 8. Documentar y probar

Incluir una coleccion `docs/requests.http` (o Postman) que ejercite los cinco requisitos, incluidos los casos 400, 401, 403 y 404. Redactar el informe tecnico en LaTeX (`docs/informe/informe.tex`) que describa las decisiones de diseno y muestre las evidencias.

4. Guia de publicacion en GitHub — criterio de evaluabilidad

El repositorio es la **fuentes de verdad** de la calificación. Debe permitir que el docente clone, arranque y verifique la API sin conocimiento previo del proyecto.

4.1 Estructura sugerida

```
inventario-mercado-<apellido>/
|-- README.md                # obligatorio: arranque + cadena de compilacion LaTeX
|-- LICENSE                  # MIT o similar
|-- .gitignore               # ignora target/, *.class, .env
|-- .env.example             # JWT_SECRET y credenciales ficticias
|-- docker-compose.yml       # app + PostgreSQL + Redis, un solo comando
|-- db/
|   |-- schema.sql           # DDL de la tabla productos y usuarios
|   |-- seed.sql             # usuarios ROLE_USER y ROLE_ADMIN (hash BCrypt)
|-- docs/
|   |-- requests.http         # o coleccion Postman
|   |-- capturas/            # 200, 400, 401, 403, 404, cache hit/miss
|   |-- informe/
|       |-- informe.tex       # informe tecnico (obligatorio, criterio P2)
|       |-- referencias.bib
|-- src/                      # codigo Spring Boot 3
```

4.2 Contenido obligatorio del README.md

Datos del estudiante; version exacta de Java y Spring Boot; requisitos previos (Docker Desktop); **arranque en un solo comando**; usuarios semilla (uno con ROLE_USER, uno con ROLE_ADMIN) y como obtener el token JWT; URL local de la API; como probar los cinco requisitos; y la **cadena de compilacion del informe LaTeX** (motor pdflatex, procesador bibtex, numero de pasadas), requisito del criterio de piso P2.

```
git clone https://github.com/<usuario>/inventario-mercado-<apellido>.git
cd inventario-mercado-<apellido>
cp .env.example .env
docker compose up -d --build
# API en http://localhost:8080/api/v1/productos
# Compilar el informe:
cd docs/informe && pdflatex informe && bibtex informe && pdflatex informe && pdflatex informe
```

5. Rubrica de evaluacion (100 %)

Cada criterio se califica en uno de cinco niveles: *Excelente* = 100 %, *Bueno* = 75 %, *Regular* = 50 %, *Insuficiente* = 25 %, *Ausente* = 0 %. La nota se obtiene como $\text{Nota}_{100} = \sum_i (\text{peso}_i \times \text{nivel}_i)$ y $\text{Nota}_{10} = \text{Nota}_{100}/10$.

Excelente = 100 % **Bueno = 75 %** **Regular = 50 %** **Insuficiente = 25 %** **Ausente = 0 %**

Criterio	Peso	Excelente (100 %)	Bueno (75 %)	Regular (50 %)	Insuf. (25 %)
C1. GET paginado con contrato uniforme {success, data, message, meta}	15 %	Endpoint devuelve productos activos paginados por page/size/sort; meta con totalElements y totalPages correctos; envoltura uniforme.	Paginacion y contrato correctos con un campo de meta impreciso o sort parcial.	Devuelve lista sin paginacion real o meta ausente.	Devuelve datos crudos sin la envoltura estandar.
C2. POST con validacion y 400 por campo	15 %	@Valid con @NotBlank, @Min y @DecimalMin; 400 con lista de errores por campo; crea el producto si la entrada es valida.	Validacion y 400 correctos con mensajes por campo poco claros.	Valida solo algunos campos o el 400 no detalla el campo.	Sin validacion en servidor; acepta datos invalidos.
C3. DELETE con soft delete y 404	10 %	activo=false sin borrado fisico; 404 con mensaje descriptivo cuando el id no existe.	Soft delete correcto con mensaje de 404 generico.	Elimina fisicamente o no devuelve 404.	El endpoint no funciona o borra la fila.
C4. Cache-aside con Redis	15 %	@Cacheable en el listado (hit/miss verificable) y @CacheEvict al crear y eliminar; sin datos obsoletos tras una escritura.	Cache operativa con invalidacion incompleta en una de las dos escrituras.	@Cacheable presente sin invalidacion (sirve datos obsoletos).	Cache declarada pero no verificable en ejecucion.
C5. Seguridad JWT: roles y 401/403	20 %	GET exige ROLE_USER, POST/DELETE exigen ROLE_ADMIN; sin token 401; rol insuficiente 403; token firmado y validado.	Roles y codigos correctos con una ruta con matiz (p.ej. 403 devuelto como 401).	Autenticacion presente pero sin distinguir roles o sin 403.	Endpoints sin proteccion efectiva.
C6. Arquitectura por capas y modelo de datos	10 %	Controlador delgado, servicio con reglas, repositorio unico acceso a datos; entidad Producto y DDL fieles al enunciado (tipos, CHECK, TIMESTAMPTZ).	Capas correctas con una responsabilidad levemente mezclada.	Logica de datos en el controlador o modelo incompleto.	Sin separacion de capas; modelo no corresponde al caso.
C7. Calidad de entrega, informe y reproducibilidad	15 %	Repo publico; README completo; docker-compose que arranca app+PostgreSQL+Redis con un comando; informe LaTeX que compila; requests.http; commits descriptivos del estudiante.	Entrega correcta con historial de commits pobre o un ajuste manual menor al arrancar.	Arranca solo con intervencion manual; informe o evidencias incompletos.	No arranca en clonacion limpia o sin control de versiones util.
Total ponderado	100 %	$\text{Nota}_{100} = \sum (\text{peso}_i \times \text{nivel}_i)$ $\text{Nota}_{10} = \text{Nota}_{100}/10$			

Penalizaciones y salvaguardas. (1) El criterio dependiente de código cuya evidencia solo aparece en el PDF o en capturas, y no en el repositorio, se califica como maximo *Regular* (50 %): el repositorio Git es la fuente de verdad. (2) La concatenacion de datos del usuario en SQL o consultas JPQL vulnerables rebaja C6 a *Insuficiente* por incumplir el control A03:2021 — Injection [8]. (3) Exponer el secreto de firma JWT en el repositorio rebaja C5 a lo sumo a *Regular*. (4) Los commits posteriores al corte declarado en la caratula PDF no se consideran para la calificacion.

6. Planilla de calificacion

Estudiante	C1	C2	C3	C4	C5	C6	C7	Nota /100

Anote en cada celda el porcentaje del nivel asignado (100/75/50/25/0). La nota aplica los pesos: C1=15, C2=15, C3=10, C4=15, C5=20, C6=10, C7=15.

Docente: Dr. Gleiston Ciceron Guerrero Ulloa, Ph.D. | gguerrero@uteq.edu.ec

Referencias

- [1] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [2] C. Walls, *Spring in Action*. Shelter Island, NY: Manning Publications, 6th ed., 2022.
- [3] M. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT),” RFC 7519, Internet Engineering Task Force (IETF), 2015.
- [4] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O’Reilly Media, 2017.
- [5] H. Washizaki, ed., *SWEBOK Guide v4.0: Software Engineering Body of Knowledge*. IEEE Computer Society, 2024.
- [6] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2012.
- [7] M. Nottingham and E. Wilde, “Problem details for HTTP APIs,” RFC 7807, Internet Engineering Task Force (IETF), 2016.
- [8] OWASP Foundation, “OWASP top 10:2021 — the ten most critical web application security risks,” 2021.
- [9] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Boston, MA: Prentice Hall, 2017.
- [10] M. Richards and N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol, CA: O’Reilly Media, 2020.